

From Bayesian Optimization to an Agentic, Claude-Driven Tuning Loop

Skipper-CCD operating-point optimization with an LTA readout

Juan Cruz Estrada Vigil

October 1, 2026

Outline

1. The starting point: Bayesian optimization of a Skipper-CCD
2. Code review: what to watch for
3. Agentic version: three cooperating agents
4. Claude as the optimizer
5. Running it in the lab: what went wrong and how it was fixed
6. Results: three campaigns on three amplifiers
7. Cost
8. Lab notebook: memory across campaigns
9. GP vs. Claude, open questions, next steps

The problem

- ▶ Skipper-CCD read out with an **LTA** controller
- ▶ Find the operating parameters that give the best images
- ▶ Each evaluation is a **real measurement**:
expose (LED) → set parameters → read out → score
- ▶ Expensive, noisy, black-box ⇒ Bayesian optimization

Parameter	Range	
V_{dd}	$[-23, -10]$	V
V_{drain}	$[-23, -10]$	V
V_r	$[-8, -5]$	V
delay_H_overlap	$[10, 30]$	clk

Optimizer works in $[-1, 1]^4$; values mapped to physical units.

The objective function

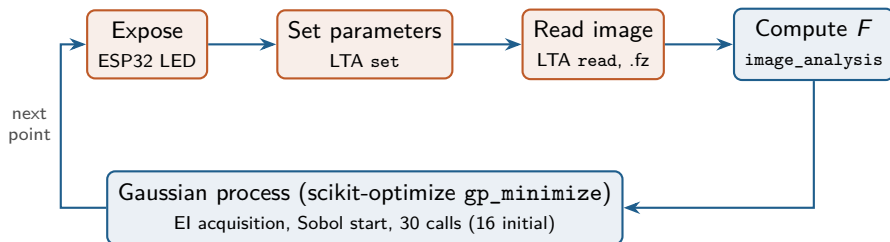
From each image: active region and overscan region

$$\text{gain} = \text{median}(\text{active}) - \text{median}(\text{overscan}), \quad \sigma_{os} = \text{std}(\text{overscan})$$

$$F = \min\left(\left|\frac{\sigma_{os}}{\text{gain}}\right|, 1000\right) + \sum_i c_i \text{penalty}_i^{p_i} \quad (\text{lower is better})$$

- ▶ With a fixed LED exposure, $\text{gain} \propto \text{ADU}/e^-$, so F tracks the read noise in electrons
- ▶ Optional penalties (negative gain, saturation, ...) are configurable; all off in the current campaign

The original loop



- ▶ Modular: `lta_control` (hardware), `image_analysis` (FITS + F), `bo_core` (optimizer, CSV, plots)
- ▶ One JSON config per campaign; warm restart from a previous run

Code review: things to watch

- ▶ **Stale images:** shell return codes ignored and the newest .fz is scored, so a failed readout would be scored using the previous image
- ▶ **Bad amplifier states can score well:** `abs()` hides an inverted gain; a clipped output has $\sigma_{os} \rightarrow 0$
- ▶ **Real example** from a lab run:

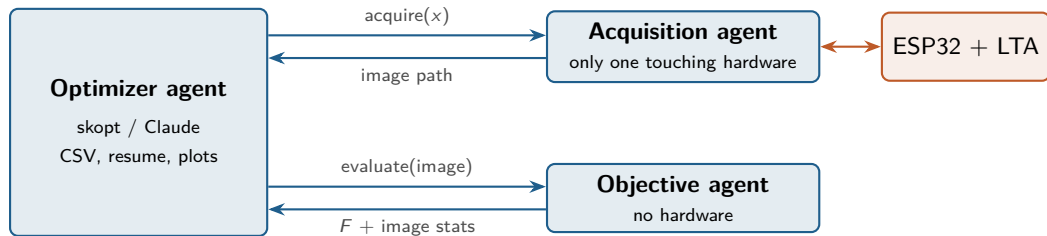
V_{dd}	V_{drain}	V_r	delay	σ_{os}	gain	F
-10.3	-12.5	-5.2	15	31924	-3544	9.0

a broken output still gets a finite, “reasonable” score

- ▶ **No settling time** after changing bias voltages
- ▶ **Noisy F :** the best *measured* point may be a lucky draw
- ▶ **Resume file** only written at the end of a run

Kept as-is on purpose (no functional changes); the resume issue was fixed later.

Step 1: three cooperating agents



- ▶ Each agent is its own process (Python multiprocessing, standard library only)
- ▶ Messages are plain JSON dicts: ready for sockets or an LLM later
- ▶ Every message is logged to `<run>_messages.jsonl`
- ▶ Same command line, same config, same output files

Messages between agents

```
{"id": 7, "kind": "acquire", "sender": "optimizer",  
  "payload": {"iteration": 3, "x_norm": [0.12, -0.40, 0.75, -0.05]}}  
  
{"id": 8, "kind": "reply", "sender": "acquisition", "reply_to": 7, "ok": true,  
  "payload": {"image_path": ".../optimize_76.fz"}}  
  
{"id": 9, "kind": "evaluate", "sender": "optimizer",  
  "payload": {"iteration": 3, "image_path": ".../optimize_76.fz"}}  
  
{"id": 10, "kind": "reply", "sender": "objective", "reply_to": 9, "ok": true,  
  "payload": {"F": 0.0296, "stats": {"noise_overscan": 636.4,  
    "charge_overscan": 5310.0, "charge_active": 26812.5, "gain": 21502.5}}}}
```

- ▶ An agent error comes back as "ok": false with its traceback; the run stops cleanly and the other agents are shut down

Validation: same results as the original

Simulated hardware for testing:

- ▶ fake `lta.sh`: synthetic images whose noise and signal depend on the voltages set
- ▶ fake ESP32 and FITS reader

Both drivers, same config and seed:

- ▶ identical `gp_results.csv`, progress CSV and image files
- ▶ identical results on warm restart
- ▶ agent failure → clean stop with the agent's error

Test suite

17 automated tests, all passing
equivalence, message log, resume, failures,
Claude optimizer (with a fake Claude),
retries, cost, notebook

Runs without hardware, network or API key:

```
python -m pytest tests
```

First lab deployment: one surprise

- ▶ Error: “*No .fz image found*”, but the LTA said “*Read done*”
- ▶ Cause: relative output path `images/...`
 - ▶ Python resolved it from *its* working directory
 - ▶ the LTA daemon resolved it from *its own* directory
 - ▶ $\Rightarrow$ images written to one folder, searched for in another
- ▶ Fix: absolute `output_base` in the config (no code change)
- ▶ Not specific to agents: the original script had the same latent issue

Step 2: Claude replaces the Gaussian process

1. **Sobol start:** 8 space-filling points (as for the GP)
2. **Each further point:** Claude gets
 - ▶ parameter descriptions and bounds, objective definition
 - ▶ *all* measurements so far, in physical units
 - ▶ image statistics (σ_{os} , σ_{act} , charges, gain), which the GP never sees
 - ▶ operator notes and the lab notebook
3. Claude returns the next point as **structured JSON**; values clipped to bounds

Drop-in replacement

`claude_minimize()` has the same interface and returns the same result object as `gp_minimize()`: progress CSV, resume, pickle and convergence plot unchanged.

```
"optimizer": {  
  "type": "claude",  
  "n_calls": 30,  
  "n_initial_points": 8,  
  "model": "claude-opus-5-5",  
  "effort": "high",  
  "notes": "..."} }
```

What Claude sees on each turn

Measurements so far (18):

#	Vdd	Vdrain	Vr	delay_H_overlap	F	noise_overscan	noise_active	charge_overscan	charge_active	gain
1	-12.0	-20.7	-7.8	17.0	2.7763	869	920.6	-1104	-791	313
...										
8	-10.3	-12.5	-5.2	15.0	15.987	879.3	898.6	-485	-540	-55
9	-14.5	-18.8	-6.3	21.0	0.04715	682.7	1769	3365	1.784e+04	1.448e+04
...										
18	-17.5	-20.2	-5.0	30.0	0.02422	685.8	2456	6107	3.442e+04	2.832e+04

Measurements left in this campaign, including the next one: 12.

Choose the next point.

Each iteration prints and logs:

- ▶ Claude proposes: Vdd=..., Vdrain=..., Vr=..., delay_H_overlap=...
- ▶ a **summary of Claude's thinking** (why this point)
- ▶ tokens used and the estimated cost

Real rows from campaign 2 (Oct 1, 2026), as sent to Claude; some rows omitted.

Running it in the lab: issues and fixes

Symptom	Cause	Fix
401 invalid x-api-key	placeholder, then a model ID, used as the key	real key from the Console; instructions clarified
Request declined: <i>reasoning_extraction</i>	prompt asked Claude to write out its reasoning in the answer	ask only for the next point; read Claude's <i>thinking summary</i> from the API
529 Overloaded after 16 images	temporary API load; SDK retried twice, then the run stopped	retries with growing waits (up to ~15 min)
16 images could not be resumed	gp_results.csv written only at the end	resume file written after every image

Lesson: with real hardware, the **surrounding software** (paths, keys, retries, checkpoints) matters as much as the optimizer.

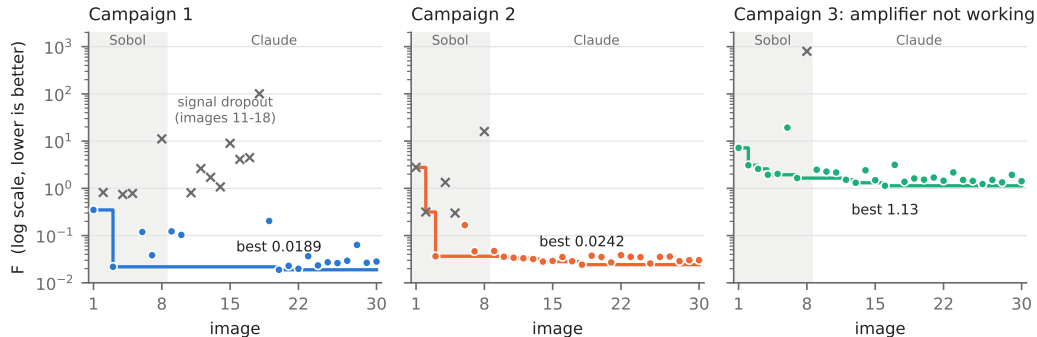
Three campaigns, three amplifiers (Oct 1, 2026)

Same config: 30 images each, the same 8 Sobol starting points, then 22 chosen by Claude (Opus 5.5); the lab notebook carried over from one campaign to the next.

	Amplifier	Best F	Best point (V_{dd} , V_{drain} , V_r , delay)	σ_{os} (ADU)	Duration
Campaign 1	[amp ?]	0.0189	(−14.5, −18.8, −6.3, 21)	≈ 700	12 min
Campaign 2	[amp ?]	0.0242	(−17.5, −20.2, −5.0, 30)	≈ 700	12 min
Campaign 3	[amp ?]	1.13	(−16.9, −19.6, −7.0, 22)	$3\text{--}4.5 \times 10^4$	11 min

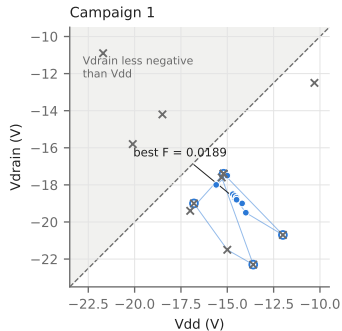
- ▶ Campaigns 1 and 2: working amplifiers, $F \approx 0.02\text{--}0.03$ in the good region; repeats at a fixed setting scatter by about ± 0.005
- ▶ **Campaign 3: amplifier not working.** Overscan noise $\sim 50\times$ higher, overscan level $\approx -4.8 \times 10^5$ ADU; F never below 1.1
- ▶ Repeatable failures: the shallow corner (−10.3, −12.5, −5.2) in all three campaigns; V_{drain} less negative than V_{dd} in campaigns 1 and 2

Performance: every image, three campaigns

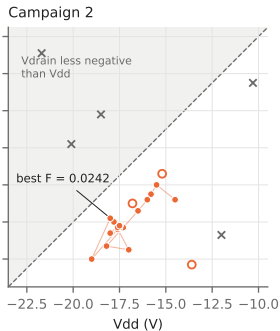


- Dots: F of each image; line: best so far; $\times$: no signal (gain < 1000)
- After the Sobol start Claude stays in or near the good region and refines: in campaign 2 every Claude-chosen image has $F < 0.05$; it also re-measures promising points to check reproducibility
- Campaign 1: a signal dropout (images 11–18, gain ≈ 0 , unrelated to the settings) was recognized by Claude and reported in the notebook

What Claude found: the $V_{dd}-V_{drain}$ relation



○ Sobol start ● Claude × no signal (gain < 1000)



- ▶ Shaded half: V_{drain} less negative than $V_{dd} \Rightarrow$ negative gain (no usable image)
- ▶ Good points: V_{drain} about 2.5–3 V more negative than V_{dd}
- ▶ Claude's own conclusion, written in the notebook:
" F is driven almost entirely by gain [...] most sensitive: the V_{dd}/V_{drain} relationship"

Why this point? Claude's thinking summaries

Real excerpts from the per-iteration log (`<run>_claude_decisions.jsonl`):

Campaign 2, image 9: using the notebook

"The notebook's recorded best is (-14.5,-18.8,-6.3,21), so I should measure that point directly to verify reproducibility and use it as a reference." (It gave $F = 0.047$, not 0.019.)

Campaign 2, image 21: reasoning about noise and boundaries

"The repeat of trial #18 showed noticeable noise (~ 0.01 variation) [...] I decide exploring V_r is valuable since we're sitting at a boundary value – I'll test V_r around -5.6 to -6.0 [...] to confirm whether -5.0 is truly optimal or just unexplored territory."

Campaign 3, image 9: noticing the broken amplifier

"These conditions look drastically different from before – noise is much higher, F values are now in the 1.6-3 range instead of 0.02-0.03 [...]"

Cost

Claude Opus 5.5 (list price)	per M tokens
input	\$4.00
output (incl. thinking)	\$20.00
cached input, read	\$0.20

Measured, 22 decisions per campaign (list-price estimate):

	tokens in	out	cost
Campaign 1	49k	9.7k	\$0.31
Campaign 2	78k	9.0k	\$0.31
Campaign 3	111k	11.2k	\$0.39

Input includes cached reads (21k, 47k, 75k), which grow with the notebook. Plus one notebook-entry request per campaign. About \$1 for all three campaigns.

Tracked automatically:

- ▶ each iteration: tokens, cost, running total
- ▶ end of run: summary
- ▶ saved in `<run>_claude_decisions.jsonl` and the result pickle
- ▶ Console Usage / Cost pages are the billed truth

Perspective

≈ \$0.01–0.02 per decision, ≈ \$0.35 per 12-minute campaign: small compared to the lab time per image.

Lab notebook: memory across campaigns

- ▶ Each API call starts from scratch: no learning between runs
- ▶ **End of campaign:** Claude writes a short entry (best region, broken settings, sensitivities, suggestions)
- ▶ **Start of next campaign:** notebook loaded into Claude's instructions as *prior knowledge*
- ▶ Current measurements take priority over old entries
- ▶ Plain Markdown file: operators read, edit, add their own notes
- ▶ Header line written by the code (date, module, best point); text by Claude

Entry 1 (campaign 1), excerpt

Measurements #11–18 all had near-zero or negative gain. This suggests an episode where the LED or signal was lost, rather than a parameter effect.

Suggestions: start at (-14.5, -18.8, -6.3, 21) [...] interleave periodic re-measurements of a reference point to detect LED or signal dropouts.

Entry 2 (campaign 2), excerpt

Last campaign's best point (-14.5, -18.8, -6.3, 21) gave only 0.047 this time. Its earlier 0.019 was probably a lucky draw or reflects changed conditions.

Use at least 3 repeats per candidate. Single-shot differences below about 0.01 are not meaningful.

Lab notebook across amplifiers: a lesson

Entry 3 (campaign 3), excerpt

Setup state changed sharply this campaign. noise_overscan was 3.1–4.5e4 ADU, about 50× the ~700 ADU of earlier campaigns. [...] F values from this campaign are not comparable with earlier entries. The likely cause is a hardware, offset or cabling change rather than a parameter effect, but this is unconfirmed.

Before optimizing, measure a reference point and check noise_overscan and charge_overscan. [...] investigate the readout chain or offset first.

- ▶ Each campaign used a **different amplifier**, but the notebook was shared and the amplifier was not in it
- ▶ Claude correctly detected that conditions had changed, and that the old best points no longer held
- ▶ It could only guess the cause (“hardware, offset or cabling”)
- ▶ The third amplifier was in fact not working: the right call was to stop optimizing and check the readout chain, as the entry says

Improvement

Record the amplifier in each entry, or keep one notebook per amplifier (“notebook” path in the config).

Gaussian process vs. Claude

	Gaussian process	Claude
Information used	parameters and F only	parameters, F , image statistics, notes, notebook
Reproducible	yes (fixed seed)	no (choices can vary)
Explains choices	no	yes (thinking summary per point)
Uncertainty model	explicit (GP posterior)	implicit
Memory across runs	warm restart (data only)	data + written lessons
Cost	free	small API cost per campaign
Failure modes	mis-fit on outliers	API availability, safety filters

Same agents, same outputs: switching is one line in the config.

Open questions and next steps

- ▶ **Noise of F :** repeat one point ~ 10 times; needed for any convergence or “which point is better” statement
- ▶ **Stopping rule:** today a fixed budget of `n_calls`; options: no-improvement rule, or Claude declares convergence
- ▶ **Head-to-head:** GP vs. Claude, same day, same budget
- ▶ **Better objective:** ask Claude (with literature search) to define “*best flat field for astronomy*” as computable criteria, reviewed by us, then computed deterministically
 - ▶ may need flat pairs / several exposure levels per iteration
- ▶ **Robustness:** API key check at start-up, flag broken images (negative gain), cheaper models (Haiku)
- ▶ **Notebook per amplifier**, and a reference-point check at the start of each campaign (both suggested by Claude’s own notebook entries)

Summary

1. Reviewed the existing Bayesian optimization of the Skipper-CCD
2. Rebuilt it as three agents (acquisition, objective, optimizer), with results verified identical to the original
3. Added Claude as a drop-in optimizer that uses the image statistics and explains every choice
4. Made it lab-robust: absolute paths, retries, per-image checkpoints
5. Added cost tracking and a lab notebook that carries lessons from one campaign to the next
6. Three 30-image campaigns in about 12 minutes each, about \$0.35 each: $F \approx 0.02$ on two amplifiers; the third flagged as not working

Code: `optimize_agents.py`, `agents/`, `tests/`